Hybrid methods for optimizing a linear fractional function over the integer efficient set:

what each hybridization is worth, measured

Abstract

Optimizing a linear fractional function Φ over the efficient set of a multi-objective integer program is usually done by repeatedly truncating the *decision* space with Sylva–Crema cuts. Each cut states a disjunction, so it costs p binary variables and $p+1$ big-M rows, and the sub-problem solved at every iteration is strictly larger than the one before it. We first move the unexplored part of the problem into *criterion* space, as a list of boxes: the truncation becomes a box subtraction, every sub-problem is the original feasible set plus a handful of ordinary linear rows, and the model never grows. On 35 instances this is faster on 34, by 5.05× overall and 9.01× on the heaviest. Re-measured on thirty instances per configuration, the margin widens monotonically in p: median ratios of 1.52×, 2.69×, 4.99×, 7.93× and 14.57× for $p = 2 \dots 6$. By $p = 6$ the two methods solve *the same number* of sub-programs and one still takes 14.57× as long, so the whole margin lies in the cost of a single sub-problem — which is the cost model above, read off the measurement rather than argued.

We then hybridize that exact search three ways and report what each is worth. Handing it the unfinished work of the published method (exact–exact) *ties* it. Seeding it from a Pareto local search with an archive (exact–metaheuristic) is the one hybridization that pays, though less than a first sample suggested: a median of 1.10× over thirty paired instances, with the seed exactly optimal on 24 of them. Seeding it from the augmented weighted Tchebychev program — the scalarization the surrounding literature generates efficient points with — **never pays**: 1.02× on the same thirty, a wash, where a smaller sample had read 0.71× and this note called it a loss.

Two further changes sharpen the exact search without altering what it returns. Pruning the open list the moment its best bound cannot beat the incumbent removes a quarter of the sub-problems and no measurable time; stacked on the metaheuristic seed it removes 4% rather than 25%, because the two feed the same bound comparison and are substitutes rather than complements. And not re-deriving a theorem of Ecker and Kouada once per inefficient box removes 6% of the branch & bound nodes at an identical box count. Both are reported in work rather than in seconds, because both effects are smaller than this machine's noise floor and no number of repeats changes that. A third, warm starting each box from its parent's basis, removes 11% of the simplex pivots at an identical box count.

Two results arrived late and are the ones we would keep if only two could be kept. Completing the front to the whole efficient set was dominated by the slice scan; *solving* the p slice equations instead of pruning on them, which eliminates that many variables before the scan starts, is worth 49× in the median and up to 130×. And a Pareto walk that hands its verified archive to the front enumeration — a seed that removes the cost of *discovering* a vector rather than the cost of proving one optimal — reaches the whole of $E(P_D)$ at 1.31× the pure exact method over 150 instances, faster on 139 of them. That hybridization pays where three earlier ones did not, for a reason the earlier failures predict: the verification its seeds need is a test the enumeration was going to pay anyway. It also had to be ordered the way the search's own probe would meet the points; in archive order the same hybrid *loses*. Against those, four separate changes that remove a fifth to a quarter of the sub-problems buy nothing at all, and we say why rather than leave a fifth attempt to someone: in this search the sub-problems that can be removed cheaply are the ones that were already cheap. The Tchebychev result is the informative one, and it does not say the

scalarization is weak. As a generator it is strictly stronger than a weighted sum, reaching unsupported efficient points no positive weighted sum can reach. It is a poor *seed* because it steers in criterion space relative to the ideal point and never looks at Φ. Symmetrically, the same free optimum that saves the box search a third of its work saves the decision-space method *zero* iterations. What a heuristic incumbent is worth is a property of the exact method it is attached to, not of the heuristic; we measure this in both directions. All arithmetic is exact and rational, every answer is proved optimal and cross-checked against independent references, and the negative results and two corrected measurement errors are reported alongside the wins.

Contents

1 Problem **3**

2 Where the cost of the decision-space method comes from **4**

3 The same truncation in criterion space **4**
3.1 How a box is written, and why not with numbers 4
3.2 The split 5
3.3 Bounding, and why it is what makes seeding possible 5
3.4 Stopping when the best bound still open is hopeless 5

4 Two obstacles specific to a fractional objective **6**

5 The algorithm **6**
5.1 Not paying twice for Ecker & Kouada 7
5.2 Paying the efficiency test less often 7
5.3 Warm-starting a box from its parent 8

6 Computational results: decision space versus criterion space **9**

7 First hybridization: exact–exact **10**

8 Second hybridization: exact–metaheuristic **10**
8.1 The heuristic 11
8.2 Where correctness lives 11
8.3 What it costs, and what it is worth 11

9 The asymmetry: a seed is worth what the exact method can do with it **12**
9.1 Two ideas that feed the same mechanism do not add 12

10 Third hybridization: the Tchebychev reference **13**
10.1 The program 13
10.2 As a generator it is strictly stronger than a weighted sum 14
10.3 As a seed it loses to not seeding at all 14
10.4 The two generators compared directly 14

11 Delivering a subset of the efficient set **15**

12 The complete front, in criterion space **15**
12.1 Is the baseline fair? Where the decision-space time actually goes 17
12.2 What the pruning is worth 18

13 From the front to the complete efficient set **18**
13.1 Solving the slice equations instead of pruning on them 20

14 Seeding the enumeration from a generator, which does not pay **21**

15 The hybridization that pays: a walk that seeds the enumeration **21**
15.1 Why a seed is worth something here at all 22
15.2 Why the archive pays where the scalarisation did not 22
15.3 The order is the whole difference 22
15.4 What it is worth, on 150 instances 22
15.5 On fractional criteria, where the problem actually lives 23
15.6 Five instance families, and a prediction that was wrong 23
15.7 Where the hybrid fails, and the line that fixes it 24

16 A wall met four times, and the one change that went round it **25**
16.1 And the same wall on the proof, not only the search 25

17 Three measurement lessons, and what they cost **26**

18 A fourth lesson, and the largest: a method is not its proof **26**
18.1 The pattern, in two halves 27
18.2 The eleven 27
18.3 What the audit did not find, and how we know 28

19 Summary of what was and was not gained **29**

20 Prior work, and what remains to be checked **29**

21 Reproducibility **30**

1 Problem

Let

$$(P_D) \qquad \text{“ max ”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \qquad x \in D = \{\, x \in \mathbb{Z}_+^n : Ax \leqslant b \,\}, \quad (1)$$

with D non-empty and bounded and every denominator $D_k(x) = d_k^\top x + b_k$ strictly positive on D. A point $x \in D$ is *efficient* when no $y \in D$ satisfies $Z(y) \geqslant Z(x)$ with at least one strict inequality; $E(P_D)$ denotes the set of efficient points. Linear criteria are the degenerate case $d_k = 0$, $b_k = 1$.

Given the decision maker's criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$, the problem is

$$(P_E) \qquad \max \ \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \quad (2)$$

$E(P_D)$ is a union of faces of D: non-convex, with no explicit description. The point of any method for (P_E) is to reach the global optimum *without enumerating* $E(P_D)$.

An efficient point is *supported* when it maximizes some strictly positive weighted sum $w^\top Z$ over D — equivalently, when its criterion vector lies on the convex hull of the criterion image. The distinction matters throughout Section 10: in an integer program unsupported efficient points are common, and no weighted sum reaches any of them.

2 Where the cost of the decision-space method comes from

One iteration of the published method maximizes Φ over a truncated region D_l, tests the maximizer x_l for efficiency, banks the best Φ on the criterion slice about to be destroyed, and then truncates:

$$\text{delete} \quad \{\, x : Z(x) \leqslant Z(\tilde{x}) \,\}. \tag{3}$$

Keeping the complement means keeping the x for which *some* criterion strictly improves on $\tilde{x}$. That is a disjunction, and a disjunction is not a system of linear inequalities. It is modelled with one binary per criterion,

$$e_k(x;\tilde{x}) \geqslant 1 \cdot y_k + M_k(1 - y_k), \quad k = 1, \ldots, p, \qquad \sum_{k=1}^{p} y_k \geqslant 1, \tag{4}$$

where M_k is a lower bound on e_k over the region, so $y_k = 0$ leaves its row vacuous and $y_k = 1$ forces a strict improvement.

Every cut therefore adds p binaries and $p+1$ rows. On the heaviest instance we ship, eleven cuts leave 33 binaries and 44 rows on top of D, and the late sub-problems cost far more than the early ones. This is the whole of the method's cost profile: the number of iterations is modest, but each one is solved on a strictly larger mixed-integer program than the last.

This observation fixes the cost model used for every comparison in this note:

$$\text{total cost} \;\approx\; (\text{number of sub-problems solved}) \times (\text{size of a sub-problem}). \tag{5}$$

The two factors are attacked by different ideas. Section 3 removes the growth of the second. Sections 8 and 10 attack the first, and the measurements there are only interpretable because the second has already been made constant.

3 The same truncation in criterion space

Equation (3) deletes a *box* from $\mathbb{R}^p$. If the unexplored part of the problem is carried as a list of boxes rather than as a region of $\mathbb{Z}^n$, the truncation is a rewrite of that list, and each sub-problem is

$$\max \; \Phi(x) \qquad \text{s.t.} \qquad x \in D, \quad \text{the box's rows}, \tag{6}$$

which is D plus at most $2p$ ordinary linear rows. No binary variable appears, and **the model never grows**: a box deep in the search carries a few more rows than a shallow one, and that is all.

3.1 How a box is written, and why not with numbers

The obvious encoding, $l_k \leqslant Z_k(x) \leqslant u_k$, fails. In the general case Z_k is a ratio, so its values are rational, and the exact "+1" step that the split below needs would be lost.

Write the box with the linear form of the paper's Theorem 4 instead:

$$e_k(x;a) \;=\; D_k(a)\, D_k(x)\big(Z_k(x) - Z_k(a)\big) \;=\; \big(D_k(a)c_k - N_k(a)d_k\big)^{\top} x + \big(D_k(a)a_k - N_k(a)b_k\big). \tag{7}$$

Both denominators being positive, e_k carries the sign of $Z_k(x) - Z_k(a)$ while being *linear in* x; with integer data it is integer-valued on integer points. Hence, exactly and with no minimal step to estimate,

$$Z_k(x) > Z_k(a) \iff e_k(x;a) \geqslant 1, \qquad Z_k(x) \leqslant Z_k(a) \iff e_k(x;a) \leqslant 0. \tag{8}$$

Linear and fractional criteria therefore go through the same code, and on a linear criterion e_k collapses to $c_k^{\top} x - c_k^{\top} a$.

3.2 The split

Proposition 1. *Let B be a box and $a \in D$. For $k = 1, \dots, p$ put*

$$B_k \;=\; B \;\cap\; \{\, x : e_k(x;a) \geqslant 1 \,\} \;\cap\; \bigcap_{j<k} \{\, x : e_j(x;a) \leqslant 0 \,\}. \tag{9}$$

Then $B_1, \dots, B_p$ are pairwise disjoint and $\bigcup_k B_k = B \setminus \{\, x : Z(x) \leqslant Z(a) \,\}$.

Proof. Let $x \in B$ with $Z(x) \not\leqslant Z(a)$. The set of indices at which x strictly beats $Z(a)$ is non-empty; let k be its smallest element. By (8), $e_k(x;a) \geqslant 1$, and $e_j(x;a) \leqslant 0$ for every $j < k$, so $x \in B_k$. Conversely a point of B_k has $e_k \geqslant 1$, hence $Z(x) \not\leqslant Z(a)$. Disjointness: membership of B_k forces k to be exactly that smallest index, and a point has only one. □

Each of the p children is the parent's row list plus at most k further rows, all of them ordinary linear inequalities in x.

3.3 Bounding, and why it is what makes seeding possible

The sub-problem (6) maximizes Φ over a *superset* of the efficient points whose criterion vector lies in the box, so its value bounds all of them. A box whose bound does not beat the incumbent is therefore discarded *whole* and never split. Children inherit their parent's value as a bound, which makes the open list a certified upper bound on everything still unfound; the method is *anytime* in the same sense, and for the same reason, as the decision-space one.

This is the single structural fact that Sections 8 and 10 rest on. A better incumbent does not merely stop the search sooner; it deletes subtrees. The decision-space method has no such mechanism, which is measured in Section 9 and is the reason the two methods respond so differently to the very same incumbent.

3.4 Stopping when the best bound still open is hopeless

The open list is kept in a heap ordered by inherited bound *descending*, and only the root — and the boxes a hybrid hands over — carry no bound; those rank first. So by the time a bounded box reaches the top, every box still open is bounded too, and if this one cannot beat the incumbent, neither can any of them. The search is finished, and the remaining list is busywork.

Measured before the change was made, on the 18 instances used throughout Sections 8–10: 198 of 733 sub-problems (27%) were solved despite carrying an inherited bound the incumbent already beat — and on *every single instance* that count equalled the number of boxes still open when the condition first fired. The waste is exactly a tail, never scattered, which is what the heap order predicts and which is why a single early exit is the whole of the fix rather than a per-box filter. Stopping there takes 733 boxes to 549, **25% fewer**, with the same answers, all still proved optimal.

It is not a speed-up, and we do not report it as one. The total is 7.44 s against 7.27 s at per-instance minima over five repeats, with per-instance ratios scattered from 0.77× to 1.36×. The change can only *remove* work, so every ratio below 1.00× is machine noise — and the noise is larger than the effect.

That is a confirmation rather than a disappointment, and it sharpens the cost model of (5). The tail boxes are the *cheapest* ones: each is killed by the root relaxation the instant it is handed the cutoff, so a quarter of the sub-problems were genuinely wasted and worth almost nothing. Where the count of sub-problems and the cost of the search come apart, it is the count that is the easier number to improve and the weaker one to quote.

4 Two obstacles specific to a fractional objective

Criterion-space search is a known family for optimizing a *linear* function over the efficient set. It does not transfer to (P_E) by inspection, for two reasons.

Remark 1 (Φ is a function of x, not of $Z(x)$)**.** *Two feasible points with the same criterion vector may have different values of Φ. Enumerating non-dominated vectors and evaluating Φ on a representative of each is therefore wrong. Inside every box a genuine fractional integer program must be solved, and before cutting on an efficient centre $\tilde{x}$ the sub-problem $Q(\tilde{x}) = \max\{\,\Phi(x) : x \in D,\ Z(x) = Z(\tilde{x})\,\}$ must be solved first, since the whole slice $\{Z = Z(\tilde{x})\}$ is efficient and is about to be removed.*

Remark 2 (Box boundaries must be integer-valued)**.** *As argued above, numeric bounds on Z_k are rational in the fractional case. Writing the boundaries through e_k restores an exact integral threshold and, as a by-product, makes fractional criteria work with no special case.*

5 The algorithm

Algorithm 1 Criterion-space search for (P_E), optionally seeded

1: $L \leftarrow \{\,\mathbb{R}^p\,\}$
2: $(x_{\text{opt}}, \Phi_{\text{opt}}) \leftarrow$ a *verified efficient* seed and its value, or $(\varnothing, -\infty)$
3: **while** $L \neq \varnothing$ **do**
4: remove from L a box B of largest inherited bound
5: solve (6) on B with cutoff Φ_{opt}
6: **if** infeasible, or its value $\leqslant \Phi_{\text{opt}}$ **then**
7: **continue** ▷ B is discarded whole
8: **end if**
9: let x_B be its maximizer
10: **if** x_B is efficient **then**
11: $\Phi_{\text{opt}} \leftarrow \Phi(x_B)$; $x_{\text{opt}} \leftarrow x_B$; $a \leftarrow x_B$
▷ x_B already maximizes Φ on its whole slice, so no Q is owed
12: **else**
13: $a \leftarrow$ an efficient point dominating x_B
14: solve $Q(a)$ and update $(\Phi_{\text{opt}}, x_{\text{opt}})$
15: **end if**
16: add to L the p children of B given by (9)
17: **end while**
18: **return** $(x_{\text{opt}}, \Phi_{\text{opt}})$, proved optimal

Termination: every pass removes at least the criterion vector $Z(a)$, and the non-dominated set of a bounded integer program is finite.

Line 2 is the whole of the hybridization interface. Everything in Sections 8 and 10 is a different way of filling it, and the correctness requirement is stated once, in Proposition 2 below: the seed must be *efficient*, not merely good.

Proposition 2 (Soundness of an arbitrary seed)**.** *If the seed x_0 in line 2 is efficient, Algorithm 1 returns the same $(x_{\text{opt}}, \Phi_{\text{opt}})$ as the unseeded run, and Φ_{opt} is the global optimum of (P_E). If x_0 is merely feasible, the algorithm may return a strictly suboptimal value.*

Proof. Φ_{opt} is used only as a cutoff, and a box is discarded when its bound fails to *beat* it. If $x_0 \in E(P_D)$ then $\Phi(x_0) \leqslant \max_{x \in E(P_D)} \Phi(x)$, so no box containing an optimal criterion vector is

ever discarded, and the incumbent is a feasible value of (P_E) throughout; the two runs therefore agree. If $x_0 \notin E(P_D)$ then $\Phi(x_0)$ may exceed the optimum of (P_E) — a dominated point can score arbitrarily well on Φ — and the box containing the true optimum is discarded by the very first bound comparison. □

This is why every seed in this note is *verified* before it is used, and why that verification, not the heuristic, turns out to be the cost of Section 8.

5.1 Not paying twice for Ecker & Kouada

When a box's maximiser x_B fails the efficiency test, the algorithm needs an efficient point to cut on in its place, and the obvious way to get one is to walk the dominance chain from the test's witness until a test returns $\theta = 0$. With *linear* criteria that walk is over before it starts, and the program that discovers this can be skipped.

Proposition 3. *With linear criteria, every optimal solution of the efficiency test applied to x^* is efficient.*

Proof. With $d_k = 0$ the test maximizes $\sum_k e_k(y; x^*) = \sum_k \big(Z_k(y) - Z_k(x^*)\big)$, a strictly positive weighted sum of Z up to an additive constant, over $R = \{\, y \in D : Z(y) \geqslant Z(x^*) \,\}$. Let y^* be a maximizer and suppose $z \in D$ dominated it. Then $Z(z) \geqslant Z(y^*) \geqslant Z(x^*)$, so $z \in R$; and $\sum_k Z_k(z) > \sum_k Z_k(y^*)$ because at least one component is strictly larger. That contradicts optimality, so no maximizer is dominated — ties included, since the argument never assumes y^* unique. □

The proposition is due to Ecker and Kouada; what is measured here is the cost of not using it. With *fractional* criteria it is false, and the proof says where: the k-th term of the objective carries a factor $D_k(y)$ that varies from point to point, the objective stops being monotone in Z, and a dominating point can score lower than a dominated one. There the walk is paid.

Checked before being relied on: 349 witnesses of failed tests across 60 linear-criteria instances, none of them dominated.

Table 1: Skipping the redundant test, over the 18 instances. The search is *identical* — same centres, same boxes, same answers — so only the price of reaching each centre moves.

	before	after	
integer programs solved	1466	1341	−8.5%
branch & bound nodes	8105	7618	−6.0%
boxes solved, optima	identical		

The programs removed are *cheaper than average* — 8.5% of the count against 6.0% of the nodes — because proving $\theta = 0$ on a point already known to be efficient is easier than searching for a dominator. The same shortcut applies to the decision-space method, which uses the same test, so the comparison of Section 6 is unaffected by it.

5.2 Paying the efficiency test less often

Splitting the work by operation puts the cost somewhere the wall-clock profile did not: the efficiency test is **52% of all simplex pivots on 146 calls**, 58 pivots each against 11.8 for a box sub-problem. The useful question is therefore not how to make the test cheaper but how to pay it less often.

When a box's maximiser x_B is inefficient, the test is what produces the efficient point to cut on. But the search already holds the points it has *proved* efficient, and asking whether one

of them dominates x_B is arithmetic rather than an integer program. One does on 21 of 146 tests (14%), and on 17% of the inefficient cases.

Table 2: Cutting on a remembered efficient point when one dominates the box's maximiser.

	before	after	
simplex pivots	16169	14602	−9.7%
boxes solved	549	534	−15
optima	identical, all proved		

The box count *falling* was not the prediction. Among the remembered points dominating x_B the highest is taken, and (9) removes more of the box the higher its centre, so skipping the test and cutting better compound instead of trading off.

The invariant this rests on. Every point the search remembers is efficient: it remembers the centres it cut on and, beside each, the maximiser of Q, which shares its centre's criterion vector and is therefore efficient whenever the centre is. One inefficient member would let the search cut on a dominated point and remove the true optimum, so this is pinned by a test rather than by an argument. Unlike the two changes above, this one deliberately *changes* which boxes are produced, so it is checked against the answer and the proof rather than against a box count.

5.3 Warm-starting a box from its parent

A box's region is its parent's plus a handful of rows, yet each box solved its root relaxation from scratch. A simplex phase I is needed whenever a row is not covered by a slack column, which every $e_k \geqslant 1$ row of (9) forces. Measured before anything was built: **4992 of 18193 simplex pivots, 27%, were phase I inside cold roots**, and 788 of 824 roots paid one — 6.3 pivots each against 2.3 for phase II.

The parent's basis stays a basis of the child once the new rows' slacks join it, so only those rows can be primal infeasible and a dual restoration repairs them. That is the argument branch & bound already uses for its own nodes, applied one level up; all it needs is the general form of the single-variable row that branching adds.

Table 3: Warm-starting each box from its parent, over the 18 instances.

	before	after	
simplex pivots	18193	16169	−11.1%
cold root relaxations	824	307	−63%
boxes, programs, optima		identical	

The ceiling was never 27%. Only a box with a parent can inherit one, and that is 63% of the roots. The rest are the efficiency tests and the Q sub-problems, which extend D rather than another box — and caching a basis for D would buy nothing, because its rows are all $\leqslant$ with non-negative right-hand sides, so its crash basis is free and phase I is skipped outright. The reachable ceiling was about 17%, of which 11.1% survives the restoration's own 1583 pivots. The gain grows with p (28% at $p = 7$ against 1–7% at $p = 3$), which is what more rows per box predicts, and two very small instances lose a little where the restoration costs more than the phase I it replaces.

517 warm starts, 0 stalls. The fallback to a cold solve never fired. It is kept regardless, because a restoration that gives up must cost time and never correctness — which is also why the box counts above are identical rather than merely equal in the answers they produce.

6 Computational results: decision space versus criterion space

All figures come from an exact rational implementation with no floating-point arithmetic anywhere, so every comparison below is between two exactly solved problems. Both methods were run on the same instances in the same process.

Table 4: Decision space versus criterion space, $p = 3$.

instance	decision space	criterion space	ratio
35 instances, total	40.65 s	8.06 s	5.05×
heaviest $n = 10$	23.47 s	2.61 s	9.01×
$n = 8$, hardest seed	2.97 s	0.45 s	6.56×
$n = 10$, easiest seed	0.36 s	0.41 s	0.86×

The box search is faster on 34 of the 35, and the single regression is the instance the decision-space method settles in three iterations, where paying 13 boxes is a net loss. Every answer agrees with the decision-space method and with an independent scan, and both are proved optimal everywhere.

Table 5: The margin against the number of criteria, $n = 6$, **thirty instances per row**. The statistic is the median of the *per-instance* ratios, with the interquartile range; a ratio of totals would be decided by the heaviest instance alone.

p	decision space	criterion space	ratio (median)	IQR	max
2	0.07 s	0.04 s	1.52×	[1.25, 1.91]	6.0×
3	0.34 s	0.12 s	2.69×	[1.87, 3.72]	10.1×
4	0.50 s	0.13 s	4.99×	[2.39, 11.62]	76.3×
5	2.19 s	0.25 s	7.93×	[3.61, 19.98]	33.8×
6	3.67 s	0.31 s	14.57×	[7.90, 25.17]	65.7×

The per-method columns are medians too. Three instances of the 150 were stopped by a 45 s budget — one at $p = 5$, two at $p = 6$ — and are excluded from the medians rather than counted as their budget. That censoring is *one-sided*: it can only make the decision-space method look better, so every margin here is a conservative one. On every instance that finished, the two methods returned the same optimum, which the campaign asserts rather than assumes.

The margin is monotone in p, and an earlier table said otherwise. An earlier version of this section reported 1.74, 2.99, 6.92, 103 and 24.3 from four instances per p, and the text went on to discuss why the curve turned over between $p = 5$ and $p = 6$. **It does not turn over.** That 103× was one instance contributing 110.95 of 112.77 seconds, and with thirty instances the progression is clean: 1.52, 2.69, 4.99, 7.93, 14.57. The old numbers were not wrong as measurements — they are what those four instances did — but they were quoted as a trend, and four runs cannot carry one. The spread remains wide, which is why the IQR and the maximum are printed next to every median.

The mechanism, measured rather than argued. Two forces grow with p and pull against each other. The split produces p children, so the box list branches wider; but the non-dominated

set grows too, so the decision-space method needs more cuts, and each cut costs it p binaries and $p+1$ rows. Counting the work separates them:

Table 6: Where the margin comes from. Ratios are decision space over criterion space, medians over the same thirty instances per row.

p	sub-programs	b&b nodes	time
2	1.72×	1.28×	1.52×
3	1.40×	1.79×	2.69×
4	1.19×	2.68×	4.99×
5	1.17×	3.16×	7.93×
6	**1.02**×	**5.37**×	**14.57**×

This is the cost model of Section 2 read straight off the measurement. At $p = 6$ the two methods solve **the same number of sub-programs** — 1.02×, a ratio that has fallen steadily from 1.72 at $p = 2$ — and yet one takes 14.57× as long. The whole margin has moved into the *cost of one sub-problem*: the branch & bound inside it explores 5.37× the nodes, because by then it carries p binaries and $p+1$ big-M rows per cut and the box search still carries none. The branching count that was supposed to work against the box list does not: it wins the sub-program count at small p and gives it back as p grows, while the other term compounds.

7 First hybridization: exact–exact

Since the published method closes in one iteration whenever the maximizer of Φ over D happens to be efficient, a hybrid suggests itself: run its loop, and if it has not closed after s iterations, hand the unfinished work to the box search. The handover is not a restart. Replaying each cut as a box subtraction (Proposition 1 applied to every box of the current list) reproduces the region the first phase reached, and the incumbent, being attained at a known efficient point, is a valid seed by Proposition 2.

Measured on 24 instances against the better of the two pure methods on each:

	total	within 15% of the better
$s = 3$	13.14 s	4 of 24
$s = 1$	8.47 s	23 of 24
pure box search	8.11 s	—
the published method	268.61 s	—

We report this as a negative result. At $s = 1$ the hybrid *ties* the pure box search — 8.47 s against 8.11 s here, and 1.47 s against 1.54 s on a family built so that Φ is easy; two differences of about 4% pointing opposite ways. The criterion-space search is what matters; this hybrid earns its place only as insurance for the one-iteration case, not as a third method that beats both. The first default we chose, $s = 3$, was measurably wrong and is reported here because it was: 13.14 s against 8.47 s.

8 Second hybridization: exact–metaheuristic

By Proposition 2 the box search will accept any efficient point as a seed, and by Section 3.3 a good one deletes subtrees. That makes a metaheuristic the natural first phase — it is cheap and it is guided by Φ, which is exactly the quantity the cutoff compares against.

8.1 The heuristic

A Pareto local search maintains an archive of mutually non-dominated points. From several random maximal starting points it walks the neighbourhood of each archive member, admitting any neighbour the archive does not dominate. The archive *is* the search: restarting from fresh random points between walks throws it away, and four times the effort did not compensate for that (obtaining 0.360, 0.280 and 0.458 of the optimum on three instances where the archive-driven version obtained the optimum).

The neighbourhood needs *swap* moves (-1 on one coordinate, $+1$ on another), not only unit steps. From a maximal point a unit step up is infeasible, and a unit step down lowers every criterion with non-negative coefficients, so the archive refuses it and the walk dies immediately: three archive points and an incumbent at 13% of the optimum. With swaps the archive holds 28–69 points and the incumbent reaches 60–100%.

8.2 Where correctness lives

The archive is filtered by dominance *among the points it has seen*, which is not the exact efficiency test. Proposition 2 makes that gap fatal if ignored, so nothing crosses unverified: the best archive member is put through the exact test, and if it fails, it is walked to an efficient point dominating it and that point's value is used instead. The heuristic decides *where to look*; it never decides what is true.

The gap is not hypothetical. Over a wide sweep, one archive member in roughly 2000 turned out not to be efficient — reproducibly, a concrete case being a four-variable instance whose archive retains $(2, 1, 1, 1)$. Rare is not never.

8.3 What it costs, and what it is worth

The cost is *not* the walk, which takes 0.01–0.08 s and is barely sensitive to its budget. We assumed the opposite and optimized the walk first; the timings then showed the total *falling as the walk's budget rose*, because a poor archive yields candidates needing longer repair chains, each one an integer program. The cost is the verification — the price of Proposition 2. Examining the single best archive member gave exactly the same incumbent as examining four on every instance tried, at a quarter to a half of the cost.

Table 7: Exact–metaheuristic hybrid, 18 instances. Same optimum everywhere, all proved optimal.

instance	exact only	metaheuristic + exact	ratio
total, 18 instances	9.50 s	7.60 s	1.25×
hardest $n = 10$	49 boxes, 3.02 s	34 boxes, 1.94 s	1.56×
$n = 8$, seed 0	16 boxes, 0.53 s	10 boxes, 0.34 s	1.53×
$p = 7$, seed 1	197 boxes, 1.36 s	120 boxes, 0.95 s	1.43×
$p = 5$, seed 1	6 boxes, 0.02 s	6 boxes, 0.03 s	0.56×

It wins where the search is expensive and loses where there was nothing to win, which is what a nearly fixed first-phase cost predicts. The *ceiling*, measured by handing the search the true optimum for free, is 1.42×; at 1.25× the heuristic realizes 88% of it.

The same verdict on thirty paired instances. Eighteen instances quoted as a total is the flaw of Section 6 again, so the campaign re-ran it paired: on each of thirty instances at $n = 6$, $p = 4$, the same search is solved three times — unseeded, Pareto-seeded, Tchebychev-seeded — and the statistic is the median of the per-instance speed-ups.

Table 8: What a seed is worth, thirty paired instances at $n = 6$, $p = 4$. The gap is $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$, zero exactly when the seed is optimal.

seed	speed-up (median)	IQR	seed gap	exactly optimal
Pareto archive	1.10×	[1.02, 1.27]	0.000	24/30
Tchebychev	1.02×	[1.00, 1.09]	0.170	8/30

The *ordering* is exactly what the smaller samples said, and it is the part that has held on every family tried: the Pareto seed is worth something, the Tchebychev seed is not. Two things do change, and both are reported because they are corrections.

First, **the Pareto seed is worth less than 1.25× here**: its median is 1.10× and its first quartile is 1.02×, so half the instances gain three percent or less. The win is real — both quartiles sit above 1 and the seed is *exactly optimal on 24 of 30* — but it is small, and it is not uniform: the worst instance runs at 0.618×, a genuine loss, because the verification of Proposition 2 is paid whether or not the bound can use what it buys.

Second, **the Tchebychev seed does not lose here; it does nothing**. Earlier families put it at 0.71–0.73× and this note called it a loss. On thirty instances of this family it is 1.02× with an IQR of [1.00, 1.09] — a wash. The verdict that survives both samples is the weaker one: *it never pays*. Seed quality explains why, and that figure is the stable one across families: a median gap of 0.170 against the Pareto seed's 0.000, and exactly optimal on 8 of 30 against 24 of 30. A generator that reaches unsupported points (Section 10) is still not a generator that lands near the maximiser of a *different* function.

9 The asymmetry: a seed is worth what the exact method can do with it

The ceiling experiment can be run on the published method too, and this is the comparison we consider the most transferable result in this note.

Table 9: The *same* free optimum, handed to each exact method at iteration 1.

exact method	effect of a free optimal incumbent	mechanism
criterion-space box search	1.42×, up to 52% fewer boxes	boxes discarded whole
decision-space cut method	**zero** iterations saved	none

The reason is structural. What the decision-space method cuts at iteration l is decided by the efficiency test applied to x_l, through (3); Φ_{opt} enters the sub-problem only as a cutoff inside a branch-and-bound that would have found the same x_l anyway. A box, by contrast, is compared against the incumbent *before* it is ever split, and failing that comparison deletes it together with every descendant it would have had.

So "hybridize an exact method with a metaheuristic" is not one question with one answer. The heuristic is identical in both rows above; what differs is whether the exact method has a mechanism through which a lower bound can remove work. Before building such a hybrid, the mechanism is what should be checked.

9.1 Two ideas that feed the same mechanism do not add

The same reasoning has a consequence that is easy to miss when improvements are collected one at a time. The early exit of Section 3.4 and the metaheuristic seed of Section 8 look independent — one prunes at the top of the list, the other starts the incumbent higher — but

they act through the *same* mechanism, the bound comparison of Section 3.3. So they should not be expected to compose, and measured, they do not:

Table 10: Sub-problems solved over the 18 instances, with each idea alone and with both.

	boxes solved	reduction
neither	733	—
early exit only	549	25%
metaheuristic seed only	562	23%
both	539	26%

Either alone removes about a quarter of the sub-problems; together they remove about a quarter. On top of the seed, the exit is worth 4% rather than 25%, because a good incumbent arrives early enough that the hopeless tail barely forms — there is nothing left to skip. They are **substitutes, not complements**.

We record this because the arithmetic of stacked improvements is where plausible reasoning goes wrong most cheaply. Two ideas add only when they feed different mechanisms; two that feed the same one are one idea, measured twice.

10 Third hybridization: the Tchebychev reference

The surrounding literature generates efficient points with the augmented weighted Tchebychev program, so a fair evaluation of the two hybrids above has to be made against it and not only against a weighted sum. We therefore implement it and measure it, rather than cite it.

10.1 The program

With z^* the ideal point, $z^*_k = \max\{Z_k(x) : x \in D\}$, weights $w > 0$ and an augmentation $\rho > 0$:

$$\min_{x \in D} \quad \max_k \; w_k\big(z^*_k - Z_k(x)\big) \;+\; \rho \sum_{k=1}^{p} \big(z^*_k - Z_k(x)\big), \tag{10}$$

which linearizes with a single continuous variable λ subject to $\lambda \geqslant w_k(z^*_k - Z_k(x))$ for each k. Each weight vector therefore costs one integer program on a model with $n + 1$ variables and p extra rows, plus p programs paid once for z^*.

Proposition 4. *For any $w > 0$ and any $\rho > 0$, every optimal solution of (10) is efficient.*

Proof. Let x^* be optimal and suppose some $y \in D$ dominated it, so $Z(y) \geqslant Z(x^*)$ with at least one strict inequality. Then $z^*_k - Z_k(y) \leqslant z^*_k - Z_k(x^*)$ for every k, so the max term does not increase; and since $\rho > 0$ and at least one inequality is strict, the sum term *strictly* decreases. The objective at y would be strictly smaller than at x^*, contradicting optimality. □

The augmentation is exactly what makes the second half of that argument available: with $\rho = 0$ the objective at y need only be no larger, and a merely *weakly* efficient point can minimize the program. Our implementation therefore refuses $\rho \leqslant 0$ and refuses a zero weight rather than returning a point whose status is in doubt. It also refuses fractional criteria, for which $z^*_k - Z_k(x)$ is not linear and the rows above are not constraints of an integer linear program; the references assume linear criteria as well.

Table 11: Reach from the same grid of 27 weight vectors, 20 random instances.

	efficient points reached	of which unsupported
weighted sum	66/175	**0/54**
augmented Tchebychev	112/175	29/54

10.2 As a generator it is strictly stronger than a weighted sum

A weighted sum maximizes only at supported efficient points. Varying w in (10) reaches unsupported ones too, and that is what the literature uses it for.
All 540 optima returned were efficient, as Proposition 4 requires. A wider sweep over 40 instances agrees: 216/369 against 135/369, 56 unsupported against 0, and 1080 optima with no inefficient one. On the reference instance of the published paper, three of the seven efficient points are unsupported; the Tchebychev program reaches all three and no strictly positive weighted sum reaches any.

10.3 As a seed it loses to not seeding at all

Table 12: Seeding the box search, 18 instances. "Exactly optimal" counts the instances where the seed already attains Φ^*.

seed	total	seed cost	exactly optimal
none (pure box search)	9.64 s	—	—
Pareto local search	**7.97** s (1.21×)	0.57 s	12 of 18
augmented Tchebychev	13.50 s (0.71×)	4.25 s	5 of 18

Both halves of the loss are structural rather than artefacts of the implementation.

Cost. It is $p + 1$ integer programs on an enlarged model. It is *not* the reference point: computing z^* took 0.01–0.13 s of the seed against 0.13–0.85 s for the scalarizations themselves, so replacing z^* by a cheaper reference would not change the verdict.

Quality. This is the real finding. Program (10) steers by criterion-space geometry relative to z^* and **never looks at** Φ. The local search of Section 8 is guided by Φ at every move. A good spread of efficient points is not a good Φ, and by Remark 1 it cannot be made into one by post-processing: Φ is not a function of $Z(x)$, so two points the generator regards as interchangeable may differ arbitrarily in the only quantity the search cares about.

We therefore keep the Tchebychev program as the reference the evaluation needs, and do not wire it into the default hybrid.

10.4 The two generators compared directly

Table 13: As generators of a subset of $E(P_D)$, 20 instances at $n = 4$.

	coverage of $E(P_D)$	time	certification
augmented Tchebychev	64%	2.1 s	every point efficient by construction
Pareto archive	99%	0.04 s	≈ 1 member in 2000 not efficient

Cheap, near-complete and uncertified against expensive, partial and certified. That is the trade the exact–metaheuristic hybrid closes: it takes the cheap, near-complete generator and buys the missing certificate with a single exact test, which is why the verification and not the walk is its dominant cost.

This also delimits the expected result of the thesis — "a subset of efficient solutions generated by hybrid algorithms". Both columns produce such a subset, and they are not interchangeable: one is a certified partial sample of the front, the other a near-complete sample whose membership is a conjecture until tested.

11 Delivering a subset of the efficient set

Everything above answers one question — which efficient point maximises Φ — and proves the answer. The work this note implements states a second one as its expected result: *a subset of efficient solutions, generated by hybrid algorithms.* That subset is what a decision maker looks at, and the machinery could already produce it without ever exporting it.

Two sources exist, and a third was measured and rejected.

Table 14: What each source contributes to a delivered subset, over 25 instances small enough to enumerate ($|E(P_D)| = 207$).

source	coverage of $E(P_D)$	cost
the exact search's own trail	51%	free
the Pareto archive, verified	98–100%	0.07 s + certification
augmented Tchebychev	39%	0.60 s

The exact search must prove a point efficient before it can cut on it, so its trail is certified and costs nothing beyond the search itself. The archive of Section 8 is what makes the subset nearly complete. And the Tchebychev generator of Section 10, certified by construction and the stronger generator in the reach experiment there, contributes *not one additional point* here — every point it reached the archive had already found. Its exclusion from the default is measured, not assumed.

Certification is the cost. The archive filters by dominance among the points it has *seen*, which is not the exact efficiency test; about one member in 2000 is not efficient. Certifying the whole archive is one integer program per member — 0.25 s against a 0.03 s walk at $n = 4$, 0.62 s against 0.11 s at $n = 5$ — and it is the same finding as Section 8's, one level up: the walk is cheap and the proof is not.

What is not claimed. This is a subset and completeness is never asserted. The coverage figures are measured against exhaustive enumeration on instances small enough to enumerate; where $E(P_D)$ cannot be enumerated, what fraction was found is unknowable, and the implementation says so rather than reporting a percentage it cannot support. The exact search's trail is also a *biased* sample — it is precisely the points the search had to cut on, so it concentrates where Φ is large. For a decision maker ranking by Φ that bias points the useful way, but it is a bias and is reported as one.

12 The complete front, in criterion space

Section 11 delivers a subset and refuses to claim completeness. That refusal is honest but it is also a limitation, and the same substitution this note is built on removes it.

Complete enumeration of the non-dominated set was already available, but in *decision* space: truncate D with Sylva–Crema cuts, pick a point, repair it, cut again. Every cut costs p binaries and $p+1$ big-M rows, so the sub-problems grow — exactly the cost profile Section 3 replaces. Keeping the unexplored region as a list of boxes instead gives the same enumeration with no binary anywhere and no model growth.

Table 15: Enumerating the whole front, six instances per row.

instance	$\lvert\text{front}\rvert$	criterion space	decision space	ratio
$n = 4$	9	0.12 s	1.37 s	11.4×
$n = 5$	30	0.63 s	129.21 s	**204**×
$n = 5$	9	0.29 s	0.81 s	2.8×
$n = 6$	6	0.08 s	0.63 s	8.0×

The ratio tracks the size of the front, not n. Every vector is a cut, and in decision space every cut adds p binaries and $p+1$ big-M rows, so a 30-vector front leaves 90 binaries on the last sub-problem while the box list never grows. The instance with the largest front is the one with the 204× gap; the two smallest fronts give 2.8× and 8.0×.

That is the same mechanism as Section 6, where the margin tracked the number of cuts rather than n — measured here independently, on a different task. *Four instances, and the spread is wide*: the mechanism explains it, but these are four runs and not a distribution.

Forty-five instances, and the mechanism measured. The campaign re-ran this over a sweep that moves $|F|$ on purpose — fifteen instances at each of three constraint tightnesses, $n = 5$, $p = 3$ — so the claim that the margin tracks the size of the front is estimated rather than asserted. Nothing was censored, and the two enumerations returned the same number of vectors on every instance.

Table 16: The front-enumeration margin against the size of the front, forty-five instances. Medians of per-instance ratios.

$\lvert F\rvert$	instances	ratio (median)	IQR	max
1–4	26	2.7×	[1.7, 3.9]	9.8×
5–12	17	7.5×	[5.2, 16.0]	23.8×
13+	2	31.4×	[24.9, 38.0]	38.0×
all	45	4.1×	[2.3, 7.6]	38.0×

The margin roughly triples and then quadruples as the front grows, which is the mechanism and not a coincidence of scale: $|F|$ *is* the number of cuts. The 13+ row rests on two instances and is quoted as such — this family at $n = 5$ tops out near $|F| = 13$, and the 204× instance above, with $|F| = 30$, lies outside it. That instance is an observed extreme, not a typical case; the typical case on this family is the 4.1× of the last row.

A ratio that goes the other way, and why it does not contradict this. Counting branch & bound nodes, criterion space explores *more* of them — the ratio is 0.29×, so about three and a half times as many — and still finishes four times sooner. Its nodes carry no binary and no big-M row, so each is far cheaper: it wins by solving many easy programs instead of few hard ones. The same count in Table 5 goes the other way at every p, because there the two methods solve nearly the same number of sub-programs and the decision-space ones are the harder. Both readings say the cost lives in the size of a sub-problem, which is exactly what the substitution removes.

12.1 Is the baseline fair? Where the decision-space time actually goes

A 204× margin invites the suspicion that the loser was handicapped, so we profiled it rather than quoting it. On the $n = 5$ instance, of 137 s spent producing 30 vectors, 137.21 s — **100%** — **is inside the probe**, over 31 calls and 14983 branch & bound nodes; the repair takes 0.09 s and adding the cuts 0.02 s. The model grows from 5 columns and 7 rows to 95 and 127, which is exactly p binaries and $p + 1$ rows per cut. The cost is therefore structural — it is the growth of the sub-problems, which is the thing Section 3 removes — and not an artefact of some inefficiency on the side of the comparison.

One candidate handicap remained. The probe only needs *some* feasible point of the truncated region, yet it maximises a linear direction; stopping at the first integer point found looks strictly cheaper. Measured, it is the opposite:

Table 17: Probing by maximising a direction against probing with a zero objective.

	cuts	B&B nodes	time
$n = 4$, maximise	9	658	1.35 s
$n = 4$, zero	9	1397	5.33 s
$n = 5$, maximise	9	372	0.79 s
$n = 5$, zero	9	2105	10.15 s

The **cut counts are identical**: the direction changes neither how many vectors there are nor the order they come out in. What explodes is the branch & bound inside each probe, by a factor of two to six, because a zero objective gives it nothing to prune with — every node's relaxation is worth 0, so no bound can discard a subtree until an integer point has been stumbled upon. The direction is load-bearing rather than decorative, the criterion-space method uses the same one, and the 204× stands.

We report this because it is a plausible-looking optimisation that makes things four to thirteen times worse, and because it was checked while suspecting the *comparison* was unfair — it was not.

Proposition 5. *The enumeration terminates, and records every non-dominated vector exactly once.*

Proof. Termination. The probe returns a point x of D inside the box, and the efficient point a it is repaired to dominates it, so $Z(x) \leqslant Z(a)$ and the split around a removes at least $Z(x)$ from that box. The unexplored region therefore shrinks on every pass, and the non-dominated set of a bounded integer program is finite.

Completeness. A non-dominated vector v leaves the region only through a split around some centre a with $v \leqslant Z(a)$. If $v \neq Z(a)$, that says a dominates v, which no non-dominated vector admits. So v leaves only at the moment it is recorded; and the loop ends only when the region is empty, so every v is recorded, exactly once since the centres are de-duplicated by criterion vector. □

Completeness is a flag, not a default. It is set only when the box list actually emptied. A run stopped by a box budget or a time budget returns what it has with the flag *false* — a front that cannot say it is whole is a subset with extra steps, and saying so is the difference between the two sections.

The front is not the set of efficient points. One non-dominated vector can be attained by several efficient points, and the enumeration keeps *one per vector*: the front is complete while the set of points it carries need not be. On random instances the distinction never appeared

— over 32 of them, 476 efficient points sat on 476 distinct vectors — but that is a property of the sample, since generic criterion coefficients make ties improbable. Constructed deliberately, with a criterion that ignores a variable, $|E(P_D)| = 6$ sits on 2 vectors and the enumeration returns 2 points of the 6.

That instance also shows what Q is for. Its slice $Z = (1, 2)$ holds three efficient points with $\Phi = 3/4$, $6/5$ and $3/2$; the repair lands on the first, and Q moves it to the third. So the largest value on the front is still the optimum of (P_E) — and without Q the answer would have been *half* of it, with the front itself still correct and the output looking entirely reasonable.

Vectors are not solutions. By Remark 1, Φ is a function of x and not of $Z(x)$, so one non-dominated vector can carry several efficient points with different Φ: enumerating the front does not by itself answer (P_E). Each vector is therefore paired with the maximiser of Φ *on its slice*, at one extra integer program per vector — the pairing a decision maker wants, and the one a plain vector enumeration cannot provide.

That pairing has a consequence worth stating, because it closes the circle of this note: **the largest value on the front is the optimum of (P_E).** The front enumeration is therefore a second, independent exact method for the problem, and the test suite uses it as a cross-check on the first.

12.2 What the pruning is worth

Two exact routes to (P_E) now exist: *prune* — stop as soon as no open box can beat the incumbent — or *enumerate* the whole front and take the best value on it. They share the box machinery but not the reason they stop, so the comparison measures the pruning itself.

Table 18: Solving (P_E) by pruning against solving it by enumeration, eight instances at $n = 6, 7$. Same optimum on every one.

	boxes	time
prune (Algorithm 1)	102	1.52 s
enumerate (Section 12)	215	2.96 s

1.94×, and half the sub-problems. That is *less* than one might expect: walking the entire front costs only twice what finding its single best point costs, which says the cutoff prunes a smaller share of the work than the ceiling experiments of Section 9 might suggest — those measure what a better *incumbent* saves, not what having one at all does.

13 From the front to the complete efficient set

Section 12 proves its vector list complete and says, in the same breath, that the *point* list is not: one vector is kept per slice. That was the last honest gap in the chain, and closing it turns out to cost almost nothing, for a reason worth stating on its own.

Proposition 6. *If v is non-dominated, then* every $x \in D$ *with $Z(x) = v$ is efficient.*

Proof. Suppose some $y \in D$ dominated such an x. Then $Z(y) \geqslant Z(x) = v$ with at least one strict inequality, which says v is dominated — against the hypothesis. So no y dominates x. □

Two consequences, and together they are the whole method. First,

$$E(P_D) \;=\; \bigcup_{v \text{ non-dominated}} \{\, x \in D : Z(x) = v \,\},$$

so the complete front plus its slices *is* the complete efficient set. Second, **not one efficiency test is paid on any slice point**: they are efficient by Proposition 6, not by examination. The expensive operation of this whole note — the efficiency test of Section 5.1 — is simply not invoked.

A slice is D plus p equalities, and nothing more. Fixing $Z(x) = v$ is linear in x *even for fractional criteria*: since $D_k(x) = d_k^\top x + b_k > 0$ on D, the equation $(c_k^\top x + a_k)/(d_k^\top x + b_k) = v_k$ clears to

$$(c_k - v_k d_k)^\top x \;=\; v_k b_k - a_k,$$

with no sign condition to check and no case split. So a slice carries no binary, no big-M, and nothing that grows from one slice to the next — the same property the box search has, arrived at a second time. The equalities are in fact the *tightest* rows in the system, which is what makes the depth-first scan over them cheap.

What it recovers, and what it costs. The family below fixes five variables that enter Z and adds f further variables that enter only their own bound $0 \leqslant x_j \leqslant 4$. Those are invisible to Z, so every slice is a grid of 5^f points and the front keeps one of them.

Table 19: The complete efficient set against the front alone. "Overhead" is the extra time over enumerating the front, bounding programs included.

f	|front|	$\lvert E(P_D)\rvert$	ratio	front	slices	overhead
0	7	7	1×	0.110 s	0.016 s	24%
0	13	13	1×	0.247 s	0.018 s	10%
1	7	35	5×	0.114 s	0.019 s	27%
1	13	65	5×	0.240 s	0.024 s	13%
2	7	175	25×	0.117 s	0.036 s	42%
2	13	325	25×	0.246 s	0.050 s	24%
3	7	875	125×	0.128 s	0.103 s	90%
3	13	1625	125×	0.259 s	0.167 s	68%

At $f = 3$ the front carries 0.8% of the efficient set — it is complete and it is missing 99.2% of the points, which is exactly the distinction Section 12 insisted on — and recovering all 1625 of them **never doubles the cost**. The scan is output-sensitive: 0.167 s for 1625 points is about 0.1 ms each, against the 7.68 ms an efficiency test costs, and there are no tests to pay.

The honest reading, which is conditional. At $f = 0$ the front *already was* the efficient set, and the 10–24% buys only the proof of that. On generic random instances this is the usual case: over 32 of them, 476 efficient points sat on 476 distinct vectors. So the value of this section is not a speed-up and is not uniform. It is that the method now *knows* which case it is in, instead of returning a point list whose completeness depended on an assumption about the coefficients. Where ties do occur — criteria that ignore a variable, repeated structure, any instance that is not generic — the front alone silently returns a fraction of the answer, and the fraction can be under one percent.

Thirty instances of the generic family, measured. The campaign confirms both halves of that reading on the family used everywhere else in this note ($n = 5$, $p = 3$). The overhead of completing the front has a median of 24.6%, an interquartile range of [21.2%, 36.0%] and a maximum of 73.7% — never a doubling, on any of the thirty. And **exactly one of the thirty had a non-singleton slice**. So on a generic instance the front really is the efficient set almost

always, and the complete enumeration almost always buys only the proof of that. The one instance in thirty is why the proof is worth buying: nothing about the front's own output says which of the two cases you are in.

13.1 Solving the slice equations instead of pruning on them

The scan above keeps all n variables and feeds the p slice equations to a depth-first search as ordinary pruning rows. That is wasteful, and the waste is the dominant cost as soon as n grows: profiling the complete enumeration at $n \geqslant 9$ put 25–30 s in the slice scan against under a second in the front enumeration, and at $n = 12$ a single slice could not be finished inside a two-minute budget.

The equations are *linear*. So solve them. Gaussian elimination in exact rationals expresses r of the variables as linear functions of the rest; substituting them into every row of the model leaves an equivalent system in $n - r$ unknowns, and the scan runs on that. Two rows per eliminated variable keep it inside its own box, so its bounds still prune. At each leaf the eliminated variables are rebuilt, and the one condition that is *not* linear — that they come out integer — is checked there. An inconsistent system settles the slice as empty without any scan at all.

The search space $(ub + 1)^n$ becomes $(ub + 1)^{n-r}$.

Table 20: The slice scan, pruning against eliminating. Fourteen instances at $n = 7 \ldots 12$; the same set of points every time.

instance	prune	eliminate	ratio
$n = 9$, $p = 3$	29.71 s	0.31 s	96.7×
$n = 10$, $p = 5$	119.47 s	0.92 s	129.9×
$n = 12$, $p = 3$	796.84 s	8.34 s	95.5×
median of all fourteen			**49.2**×

Exactness is untouched: every coefficient is a rational and the elimination is exact, so this changes the order of the search and nothing else. It is also the *only* change in this note that ever beat the wall of Section 16, and it did so by attacking an exponent rather than a count.

A sign error that ran, and returned nothing. The first draft had a plus where the eliminated variables' contribution must be subtracted. It ran without error, looked like a working optimisation, and returned the *empty* set on 179 of 185 slices — it would have dropped efficient points from $E(P_D)$ silently. Only the comparison against the un-eliminated scan caught it, which is what that comparison is for, and the test suite keeps it.

What is verified. The complete set is checked against exhaustive enumeration on two families, one generic and one built to force ties (24 instances, exact match, 21 non-singleton slices). Proposition 6 is checked directly rather than assumed: 74 points on non-dominated slices, every one efficient. The front's own representative is checked to lie inside its slice. And, as everywhere else in this note, an exhausted budget returns `complete = False` rather than a set that claims what it did not prove.

14 Seeding the enumeration from a generator, which does not pay

Section 10 found the augmented Tchebychev program a strong *generator* and a poor *seed*, and blamed the second on its never looking at Φ. Enumerating the front has no Φ to look at: the target is the vectors themselves, so the reason it failed there does not apply. It is also the natural candidate, because its optima are efficient *by construction* — no verification needed, which is what the metaheuristic seed pays for.

The arithmetic to check, stated before measuring: seeding k vectors removes k probes and k efficiency tests, and costs p programs for the ideal point plus one per weight vector.

Table 21: The front enumeration seeded from the Tchebychev generator, against the same enumeration unseeded. 120 instances, $n = 5 \dots 7$, $p = 3, 4$; identical fronts throughout.

instance	grid	seeded	programs	time
$n = 5$, $p = 3$	narrow	75% of front	1.06×	1.44×
$n = 5$, $p = 3$	wide	**100**%	1.48×	**2.61**×
$n = 6$, $p = 4$	wide	71%	1.21×	1.80×
$n = 7$, $p = 3$	narrow	38%	1.03×	1.16×

Every ratio is the wrong way, and the decisive row is the second: the wide grid seeds **the entire front** and is the *worst* of the eight configurations. More seeding is more loss, which is not what a useful seed does.

The reason is structural. Seeding k distinct vectors saves at most k probes and costs at least k scalarisations, plus p for the ideal point, plus one for every weight that lands on a vector already seen. And a scalarisation is not cheaper than a probe:

Table 22: One probe against one scalarisation, on identical instances.

	time	b&b nodes
one probe	9.48 ms	0
one scalarisation	20.90 ms	21

The probe maximises a linear direction over D plus a few ordinary rows and usually needs no branching at all; the scalarisation minimises a *maximum*, which costs a continuous variable and p extra rows, and it branches. It is cheaper than a probe on only 7 of 40 instances. Even crediting the seed with the 7.68 ms efficiency test it also removes — 17.2 ms saved against 20.9 ms spent — a single seed is already a losing trade before the ideal point and the duplicate weights are counted.

And the saving is not guaranteed either. Seeding was expected to remove exactly one probe per seed, and in the median it does: +1.00 over 152 instances. But the pre-split imposes a box structure the search would not have chosen, and on **5 of those 152** the seeded run spent *more* probes, up to four more. That observation is what Section 15.3 turns into a fix.

15 The hybridization that pays: a walk that seeds the enumeration

Until this section nothing in this note was both *hybrid* and *complete*. $E(P_D)$ was reached by a purely exact method; every hybridization had been attached to the optimisation search, and

the one attached to the enumeration — a Tchebychev generator, Section 14 — lost. That gap was real, and this closes it.

15.1 Why a seed is worth something here at all

A seed for the optimisation search feeds one mechanism: the *bound*. A search that already knows a good value prunes boxes whose bound cannot beat it. The front enumeration **has no bound and no incumbent** — it must exhaust every box — so a seed of that kind is worth exactly nothing.

What it does have is a different mechanism. Every box costs one probe, one efficiency test and sometimes a repair, all spent to find *a vector*; a vector already known needs none of them. Seeding here is not "start from a better value" but "do not pay to discover what you already know". Soundness is unchanged, because a seed enters the box list exactly where a found point would have: the split around it is the same split, and Proposition 5 applies verbatim.

15.2 Why the archive pays where the scalarisation did not

Two things separate them, and both are measured.

Cost per vector. A Tchebychev scalarisation costs 20.90 ms and buys one point; a Pareto walk costs about 0.011 s and buys a whole archive, covering **95% of the front** in the median.

The verification is free here, and only here. The walk proves nothing, so every archive member has to be put through the exact efficiency test — and that verification was the cost that sank the metaheuristic seed in Section 8. Here it is not a cost at all, because **the enumeration was going to pay the same test anyway**: every probe the seed removes was going to be followed by one. The verification cancels and the probe is the gain. The Tchebychev generator has the opposite profile — its points need no verification, and it has no cheap way to produce them.

15.3 The order is the whole difference

Handed over in archive order the hybrid *lost*, at 0.81×, and the probe count went *up* on 4 of 12 instances: the pre-split was building a box structure the search would not have chosen. The probe maximises a fixed linear direction, so the search splits first around whatever scores highest on it. Sorting the seeds by that same direction reproduces that structure instead of fighting it.

Table 23: Probes against the unseeded search, by the order the seeds are handed over.

order	probes	raised the count on
archive order	0.89×	4 of 12
probe order	**0.70×**	**none**
reversed	0.94×	5 of 12

The reversed arm is what makes the direction causal rather than a coincidence. **A 1.33× win was sitting behind a sort.**

15.4 What it is worth, on 150 instances

The answer is identical on every one of the 150 — the same front, the same $E(P_D)$, the same optimum of (P_E) — which the campaign asserts rather than assumes. The heuristic decides only how much of the front has to be discovered.

Table 24: The hybrid against the pure exact methods, thirty instances at each of five configurations, $n = 6 \ldots 9$. Medians of per-instance ratios, with the interquartile range.

	median	IQR	faster on
front enumeration	1.33×	[1.20, 1.51]	139/150
$E(P_D)$ complete	**1.31**×	[1.17, 1.47]	139/150
probes	0.72×	[0.67, 0.75]	*never worse*

This is the fourth hybridization in this note and the first to pay outside the optimisation search. The rule the other three established survives it intact, with the second half now doing the work: *a hybridization pays only where the exact method has a mechanism the first phase can feed* ***more cheaply than the exact method feeds it itself****.*

15.5 On fractional criteria, where the problem actually lives

Every figure above uses *linear* criteria, which is a gap worth naming in a note about ratios. It has a reason — the Tchebychev comparison needs them, since $z^*_k - Z_k(x)$ is not linear when Z_k is a ratio — but the Pareto walk has no such requirement, so the hybrid can be measured where the problem statement lives.

Table 25: Genuinely fractional criteria, ten instances per row.

configuration	$\lvert F\rvert$	cover	ratio
$n = 5$, $p = 2$	7	100%	1.54×
$n = 6$, $p = 3$	22	100%	1.35×
$n = 7$, $p = 3$	36	100%	1.25×
all 40			**1.39**×, faster on 34

At least as good as the linear case, with the walk covering the front completely. Nothing in the construction distinguishes the two: the rows (7) are integer-valued for ratios as much as for linear criteria.

15.6 Five instance families, and a prediction that was wrong

Everything in this note comes from one pseudo-random generator, and the standing caveat has been that the magnitudes belong to it. Four further families move what the mechanism depends on: `loose` raises $|D|$ by an order of magnitude while leaving $|F|$ alone; `knapsack` uses two capacity rows with all coefficients positive; `conflicting` gives the criteria opposing signs, making $|F|$ large; `aligned` correlates them, making it small.
Every family pays, 1.14–1.37×, so the advantage is not an artefact of the generator it was found with. That is what this was run for.

And it refutes a prediction we wrote down first. We expected `conflicting` ($|F| \approx 107$) to be the family the hybrid liked *most* — a large front is much to discover, and discovery is what a seed removes — and `aligned` ($|F| = 6$) the least. **The opposite holds.** `conflicting` is the worst of the five, and the margin narrows with $|F|$ inside every family as well: 1.37 → 1.30, 1.29 → 1.23, 1.33 → 1.27, 1.31 → 1.14.

What bounds it, and what does not. A seed removes *one* probe, and the enumeration spends 2.3 to 7.5 probes per vector — the rest go to boxes that turn out empty, which no seed

Table 26: Fifteen instances per cell, 150 in all. Identical front and identical optimum on every one.

family	$\lvert F\rvert$	cover	ratio	faster on
`mixed`	8 / 30	100 / 97%	**1.37** / 1.30×	28/30
`loose`	9 / 44	83 / 86%	1.29 / 1.23×	28/30
`knapsack`	12 / 41	100 / 90%	1.33 / 1.27×	29/30
`conflicting`	107 / 105	100 / 93%	1.14 / 1.14×	26/30
`aligned`	6 / 11	100 / 84%	1.31 / 1.14×	25/30
all 150			**1.23**×	**136**

can touch. So the *probe-count* saving is capped at one over that figure, by construction, and shrinks as it grows. Sweeping p at $n = 7$, twelve instances each:

Table 27: Probes per vector is a deterministic count; time is not.

p	$\lvert F\rvert$	probes / vector	probes kept	time
2	4	2.3	—	1.38×
3	8	3.0	0.68	1.48×
4	16	4.0	0.75	1.33×
5	30	4.9	0.82	1.21×
6	38	6.4	0.87	1.24×
7	54	7.5	—	1.10×

The probe column is monotone and certain: 32% removed at $p = 3$, 13% at $p = 6$. **It does not bound the time.** A seed removes a probe on a *non-empty* box, which carries an efficiency test behind it, while the probes left behind are mostly on empty boxes and are refuted at once — so the work removed is dearer than the average probe. Measured, 1/(probes kept) and the time ratio agree only loosely and at $p = 6$ the time ratio exceeds it. An earlier draft of this note turned the probe bound into "a hard ceiling of roughly 1.4×"; that is withdrawn.

What does grow with $|F|$ is the walk's cost, one efficiency test per archive member. A large front buys no more and costs more — the ordering in the table, and the same mechanism as Section 15.7 from the other side.

15.7 Where the hybrid fails, and the line that fixes it

At $n = 12$ the margin falls to 1.00×, and at $n = 12$, $p = 5$ to 0.97×. We first blamed the probe count outgrowing $|F|$. **The measurement refutes that**: probes per vector are flat across the range — 2.4, 2.7, 2.7, 2.8, 2.8, 3.0, 3.9, 5.2. What collapses is the share of the front the walk finds, and the speed-up tracks it line for line: 87% at $n = 6$, 79% at $n = 9$, 65% at $n = 10$, 35% at $n = 12$, 12% at $n = 12$, $p = 5$. A fixed budget of 8 restarts and 4000 neighbours is a fixed amount of work against a target that grows — and 4000 was chosen at $n = 7, 8$ and generalised.

Widening the walk alone confirms it and names the knob: at $n = 12$, going from 8 to 16 restarts takes coverage from 35% to 95% and the ratio from 1.03× to 1.61×, while the walk's own time barely moves. So the first round is sized to the instance, at $\lceil n/4 \rceil$ times the restarts. Over 120 instances spanning $n = 6 \ldots 12$ that gives a median **1.36**×, faster on 116.

But coverage is not the objective, and that bounds the idea. On the same $p = 5$ instance, 46% **coverage gives** 1.20× **and** 93% **gives** 1.12×: buying the last half of the front costs more walking than the probes it removes. A walk that restarts in doubling rounds until

one finds nothing new takes coverage there from 45% to 97% and is *slower* — 1.02× against 1.14×, and a median 1.32× against 1.36× over the 120, winning 109 against 116. It is kept in the code and switched off.

A correction to our own reading. The collapse was first reported here as "35% coverage, 1.00×" from *one* instance; over six seeds of that configuration the unmodified walk averages 78% and 1.35×. Severe where $|F|$ is large — 45% at $|F| = 302$ — but not as general as one instance made it look. That is the fifth time in this note that a single instance misled, which is why the campaign exists.

16 A wall met four times, and the one change that went round it

Four separate changes in this note remove between a fifth and a quarter of the sub-problems and buy nothing.

Table 28: Four attempts on the sub-problem count.

change	sub-problems removed	cost
early exit on a hopeless bound	25%	no measurable time
free contradiction filter	24.8%	0.98× pivots
range filter (ideal + anti-ideal)	+11.3%	1.03× pivots
dominated-whole filter	8–23%	1.077× pivots

Only the second is kept, and it is worth 2%. The reason is the same every time, and after four it is a property rather than an accident:

> **In this search, removing sub-problems does not remove time, because the ones that can be removed cheaply are the ones that were already cheap.**

An empty box — and 73% of them are empty — is refuted by the simplex almost at once: every probe in the families measured here costs *zero* branch & bound nodes, because the relaxation is integral or infeasible at the root. A filter that catches empty boxes is therefore catching the cheapest work in the search.

The only change that ever went round this wall is Section 13.1: it attacks an *exponent* rather than a count, and it is worth 49×. We state the wall plainly because the fifth attempt is not worth anyone's afternoon.

16.1 And the same wall on the proof, not only the search

One more attempt deserves its own paragraph, because it tests something wider than an optimisation. After the walk seeds 95% of the front, 93% of the remaining boxes are empty and the enumeration is almost purely a *proof of emptiness*. But "is there a feasible point not dominated by any of the k known ones?" is **one** question, and the box list answers it by splitting it into hundreds. Sylva–Crema answers it in one integer program: k cuts, built once and never iterated — which removes the growth that made that formulation expensive in Section 2.

It is sound: 13 of 30 instances were proved complete by a single program, with no wrong verdict. It is also 4.5× slower in the median and 100× slower at worst (239 s against 1.95 s), because k cuts mean kp binaries in one model — 88 of them at $k = 22$, $p = 4$.

That closes a question this note had left open. The claim so far was that criterion space is the better *search*, because the model never grows. The certificate tests whether it is also the

better *proof*, where there is no iteration and no growth at all. **It is, by** 4.5×: answering 1363 cheap questions beats answering one question with 88 binaries. The box decomposition is not only a better search; it is a better proof.

17 Three measurement lessons, and what they cost

We report these because the direction of a result is worth less than the knowledge of how it was checked.

An unsound quality metric. Seed quality was first scored by the ratio $\Phi(\text{seed})/\Phi^*$. That ratio *inverts when* Φ^* *is negative*, which it is on several instances in these families: a strictly worse seed can score above 1. All seed-quality figures in this note use the non-negative gap $(\Phi^* - \Phi(\text{seed}))/|\Phi^*|$, which is well defined whatever the sign and is zero exactly when the seed is optimal. No published number changed direction under the correction, but the metric was wrong and one table read 1.286 for a seed that was in fact 29% below the optimum.

Seed quality is not seed value. On a looser instance family, the Pareto seed is *exactly optimal on 15 of 24 instances* and still buys only 1.05×, because that search is not bound-limited: there are few boxes whose bound the incumbent can defeat. Quality is a property of the heuristic; value is a property of the pair. What is stable across all families we tried is the *ordering* — Tchebychev loses to no seed at all (0.73×, 0.91×), the Pareto seed ties or wins — not the size of any win. Single speed-up figures quoted without their instance family should be read with this in mind, including ours.

Below the noise floor, count work and not seconds. The change of Section 5.1 removes 125 integer programs, and its wall-clock A/B reads 0.96× — *slower* — faster on 9 of 18 instances. That cannot be causal: removing work cannot cost time. The machine used here has a noise floor of roughly ±30% per instance even at per-instance minima over five repeats, so it cannot resolve a 6% effect, and no number of repeats fixes that. Counting integer programs and branch & bound nodes gives the same figure on every run and every machine, which is why Section 5.1 reports those and no seconds at all.

Two habits follow, and we adopted both only after failing at them. A measurement taken while anything else was running on the machine is discarded, not quoted — the first A/B for that section was contaminated exactly so and was re-run. And a gain predicted from the *mean* cost of a call is not a prediction: the profile that motivated the change showed the efficiency test at 18.4 ms per call, the dearest operation in the search, but what the shortcut removes is the repair's test at 7.68 ms, the cheap kind. Reasoning from the wrong row of the right profile overstated the gain by half.

18 A fourth lesson, and the largest: a method is not its proof

The three lessons above are about measuring. This one is about what was being measured, and it is the reason the section exists rather than a footnote: we audited the implementation against its own stated hypotheses and found *eleven* places where a hypothesis asserted in this note was never tested in the code. All eleven are fixed and carry regression tests, and no measured result moved. We report them at this length because the pattern behind them generalises past this method, and because a reader entitled to doubt the ratios in Sections 8–15.6 is entitled to know how they were defended.

18.1 The pattern, in two halves

A proof's conclusion computed from control flow. Every flag in the implementation claiming something had been *proved* was computed from the shape of the loop — it finished, no sub-problem remained — and not from the hypotheses the proof needs. "The box list is empty" was reported as "the front is complete", which is sound exactly when D is bounded and every D_k is strictly positive on it. Those two facts are stated in Section 1 as assumptions and were, until the audit, assumed in the strict sense: nothing established them. The same substitution had appeared independently in the front enumeration, in both optimisation searches, in the completeness certificate of Section 16, and in the two reference oracles.

Neither failure is loud. On $D = \{x \in \mathbb{Z}^2_+ : x_1 - x_2 \leqslant 0\}$ — feasible, with efficient points, but unbounded — the enumeration returned an *empty front marked complete* and both searches reported *infeasible*. On $Z_1 = x_1/(x_1 - 1)$ over $0 \leqslant x \leqslant 3$, whose denominator changes sign, it returned a single vector as the whole front while a feasible point dominates it.

Every saving was a trust assumption. This is the half that generalises. Each parameter added to the API in order to *avoid recomputation* — a precomputed variable box, a precomputed front, a set of seeds, an incumbent together with its value — is a way for a caller to supply a hypothesis without proving it. Each was introduced as an optimisation; each silently became an assumption the code depended on. A caller passing a variable box is asserting that the box contains D, and nothing downstream can detect a box that does not, because every routine that receives one searches it honestly and reports a complete-looking answer about a feasible set that is not D.

The practical form of the lesson: a parameter that lets the caller skip work the callee would otherwise do is a parameter that lets the caller skip a *proof* the callee would otherwise have. The two are the same parameter, and we added five of them without noticing.

Why the tests could not see any of it. All 93 tests then in the suite fed *valid* input. They established that the method computes the right answer when its hypotheses hold — which was never the question — and they could not establish that anything enforced them. A suite of that shape is blind to this entire class by construction, and the blindness is invisible from inside it: every test passes, on every commit, for years. The suite now holds 103 tests, and the ten added all feed deliberately invalid input and require a refusal.

18.2 The eleven

Table 29 lists them. Findings 10 and 11 are the two that sit in the validation apparatus rather than in the method, and they are the serious ones.

Finding 10 is in the completeness certificate of Section 16, which searches $\{x \in D : \Phi(x) >$ value$\}$ inside a caller-supplied box. A box smaller than D hides challengers, and the certificate then returns *valid* on a point that is not optimal: with the box $[1, 1]$ it certified an efficient point at $\Phi = 1/9$ while the true optimum is $5/17$. That is the worst location in the package for such a defect, because the certificate is what validates answers past the size at which brute force reaches — a false proof is not a wrong answer but a wrong *check* on other answers.

Finding 11 is the same defect one level further out, in the two reference oracles themselves, which between them are called from about 45 sites across the tests and the examples, each passing a box written by hand. Handed a box smaller than D neither failed nor returned less with a warning: they answered confidently about the efficient set of a different feasible set, under the names $E(P_D)$ and "the exact optimum of (P_E)". On $x_1 + x_2 \leqslant 6$, $x_1 - x_2 \leqslant 2$, whose efficient set holds five points and whose optimum is $2/5$, the box $[1, 1]$ yielded one efficient point reported as $E(P_D)$, and $1/3$ reported as the exact optimum. A clipped oracle does not produce a wrong answer; it produces a wrong *standard*, and a method measured against it looks correct

precisely when it agrees with the clipping. Every ratio in this note is a ratio against one of these.

Table 29: The eleven findings. "Silent" is meant literally: none raised, none returned a warning, and none was detectable by any caller downstream.

	hypothesis never established	silent consequence
1	D bounded	empty front marked *complete*; both searches reported *infeasible*
2	$D_k(x) > 0$ on D	one vector returned as the whole front while a feasible point dominates it
3	a front handed in belongs to this problem	a foreign front completed into a point set called $E(P_D)$
4	the variable box contains D (slice enumeration)	clipped slices lose efficient points
5	Φ is defined on every slice	the point list and the value list fell out of step
6	a handed-over incumbent is feasible	the search started below a bound no point can reach
7	a handed-over incumbent's value is its own Φ	the search pruned against a bound no point realises
8	seeds are efficient	inefficient points entered the box list unprobed, against Proposition 2
9	the returned point is efficient when state was handed over	an inefficient point returned as the answer to (P_E)
10	the certificate's box contains D	a suboptimal point certified *proved*
11	the reference oracles' box contains D	$E(P_D)$ and $\max\Phi$ redefined; agreement with them became uninformative

18.3 What the audit did not find, and how we know

The mathematics survived unchanged. The four propositions the method rests on were re-derived and re-tested: the slice characterisation of Section 13, the completeness of the front enumeration, the disjointness of the p-way split (Proposition 1), and the validity of the big-M in (3). The last was additionally checked numerically on 220 point pairs with no violation and with the bound exactly attained, which is what makes it the smallest valid choice rather than merely a safe one. All eleven findings were in the *enforcement* of hypotheses and none in the reasoning that uses them.

Three observations, stated as observations because the discipline of the audit is not to assume exactly this kind of thing:

- Every figure in this note was produced by passing the box the model derives for itself, never a hand-written one, so none of them was affected.
- With the finding-11 check active the suite passes 103/103 and the four examples that use the oracles all run clean. That is the evidence — not the assumption — that none of the roughly 45 boxes written by hand across the tests and examples was ever clipping D, and that the two boxes used for the worked examples ($[6, 12]$ and $[3, 3, 3]$, against derived bounds $[3, 3]$ and $[3, 2, 2]$) are valid enclosures. The trap sat in the API and never in a reported measurement.

- The scaling study still certifies every instance from $n = 10$ to $n = 20$ after finding 10 was closed, so the guard did not invalidate the study that rests on the certificate.

The guards are permanent rather than a debug mode, because the cost is a fixed charge per problem and not per sub-problem: boundedness is one integer program for the whole problem; the denominators one program per *genuinely fractional* criterion and none at all for a linear one; the variable box n programs, cached on the problem, so every entry point can afford to re-ask; verifying a seed is the exact test Proposition 2 already required before the seed could be trusted. Against the sub-problem counts reported throughout this note they do not register.

Remark 3. *The honest reading is the one in this section's title. Each of the eleven was a place where the implementation claimed a proof's conclusion while skipping its premise, and none would have been caught by testing that the method gets the right answer on valid input. They were caught by asking, for each hypothesis stated in this note, which line of code establishes it. Eleven times the answer was none.*

19 Summary of what was and was not gained

Table 30: Every hybridization attempted, with its verdict.

hybridization	verdict	effect
decision space → criterion space	**win**	1.5× at $p = 2$ to 14.6× at $p = 6$
exact–exact handover ($s = 1$)	tie	8.47 s vs 8.11 s
exact–exact handover ($s = 3$)	**loss**	13.14 s vs 8.47 s
exact–metaheuristic (Pareto archive)	**win**	1.10× median, 24/30 seeds exact
exact–Tchebychev seed	**loss**	1.02×: a wash, never pays
Tchebychev as a generator	**win**	reaches unsupported points
incumbent seeding in decision space	**loss**	zero iterations saved
early exit on a hopeless bound	partial	25% fewer boxes, no time
early exit *on top of* the seed	**loss**	4% fewer boxes: a substitute
skipping the redundant test	**win**	6% fewer nodes, same search
warm-starting a box from its parent	**win**	11% fewer pivots, same search
cutting on a remembered efficient point	**win**	10% fewer pivots, 15 fewer boxes
zero-objective probe (a baseline check)	**loss**	4–13× slower, same cuts
slices instead of an efficiency test	**win**	the whole of $E(P_D)$, under 2×
eliminating the slice variables	**win**	49× median, up to 130×
Tchebychev seeding the enumeration	**loss**	worse the more it seeds
Pareto walk seeding the enumeration	**win**	**1.31**× to all of $E(P_D)$
free box filters (three of them)	partial	2%, then two losses
one-program completeness certificate	**loss**	4.5× slower, sound

Of the nineteen, nine are wins, eight are losses, one is a tie and two pay in sub-problems but not in time. They are reported at the same length as the wins because they carry the same information: a hybridization pays only where the exact method has a mechanism the first phase can feed, that mechanism is cheaper to identify than the hybrid is to build and measure, and two ideas feeding the same mechanism are one idea counted twice.

20 Prior work, and what remains to be checked

Criterion-space search for optimizing a function over the efficient set of a multi-objective integer program is an established family, and the surveys report it as currently more effective than

decision-space search. Those methods are for *linear* objectives. The literature on a linear *fractional* objective over an integer efficient set — including the branch-and-cut methods this note compares against, and the extensions to fractional criteria and to quadratic objectives — works in decision space. The augmented weighted Tchebychev program of Section 10 is standard and is used here exactly as the references use it; nothing about it is claimed as new. The combination treated in Section 3 appears not to have been addressed, for the reason given in Remark 1: Φ is a function of x and not of $Z(x)$, so the transfer is not immediate.

This is a statement about what we did not find, not a claim of novelty. A proper bibliographic review, including the sources behind paywalls, remains to be done before any such claim is made.

21 Reproducibility

The implementation is pure Python with no third-party dependency, computes in exact rational arithmetic throughout, and carries 103 tests. Several check properties rather than answers, since a method that silently lost efficient points would still agree with the optimum on favourable instances:

- Proposition 1 directly — for every centre and every feasible point, the point is either removed by the cut and in no child, or kept and in exactly one.
- Proposition 2 operationally — the metaheuristic is checked, over 14 instances and against an independent scan, never to hand over an inefficient point.
- Proposition 4 — every optimum of (10) is checked efficient against exhaustive enumeration, and the refusals ($\rho \leqslant 0$, a zero weight, fractional criteria) are checked to fire.
- Proposition 3 — every witness of a failed test on linear criteria is checked both efficient and dominating; the shortcut is checked to return exactly what the walk it replaces returns, so the centre never moves; and on fractional criteria the walk is checked to still be taken and still to end efficient.
- The early exit of Section 3.4 — the run must end with boxes still open and unsolved while staying proved optimal and agreeing with the independent scan, and a seeded run, where the exit fires sooner, must return the same value as an unseeded one.
- Proposition 5 — the enumerated front is checked against exhaustive enumeration, not a sample of it; each vector's point is checked efficient and checked to attain that vector; the largest "Phi" on the front is checked to equal the optimum of (P_E); a budget-limited run is checked to refuse the completeness flag; and the whole of it is checked again on fractional criteria, where the centre comes from a dominance walk rather than from Proposition 3.
- The deliverable of Section 11 — every member is checked efficient and checked to carry the value Φ actually takes there, the proved optimum is checked to be present and ranked first, and each source is checked to work alone.
- The invariant of Section 5.2 — every point the search remembers is checked efficient, since the shortcut there cuts on one without re-testing it.
- Section 5.3 — a row added to a solved tableau is checked against rebuilding the model and solving it cold, on 120 random programs; and a child box's warm root is checked to reach the same relaxation value as its cold one.
- The unsupported-reach claim — on the reference instance, the three unsupported efficient points are all reached by (10) while no positive weighted sum reaches any.

The ninety-three tests that predate the audit, the ones listed above among them, all feed *valid* input — which is the limitation Section 18 is about: together they establish that the method is correct when its hypotheses hold, and none of them can establish that anything enforces them. Ten further tests were added for that, and they are of the opposite shape — each feeds input that violates a stated hypothesis and requires the call to refuse rather than answer:

- An unbounded D and a sign-changing denominator are each checked to raise at every exact entry point, rather than to return the empty complete front and the dominated single-vector front they used to.
- A variable box that does not enclose D is checked to be refused by the slice enumeration, by the completeness certificate and by *both reference oracles* — four separate routines that each took it on trust — with the valid box checked to still give $|E(P_D)| = 5$ and $\Phi^* = 2/5$ on the same instance.
- A front belonging to another problem, an infeasible incumbent, an incumbent whose stated value is not its own Φ, and an inefficient seed are each checked to be refused; and the optimisation is checked, on exit, to re-test the point it returns whenever state was handed in.
- A slice on which Φ is undefined is checked to leave the point list and the value list in step, over 201 instances, 53 of which produce such a slice.

Every comparison in this note is reproduced by scripts in the repository, run on every commit by continuous integration, which assert that each seeded run returns the same proved optimum as the unseeded one.